Gmail Jerry Lin <jerry@museder.com>

[WordPress Plugin Directory] Review in Progress: Museder RestoreOne

Jerry Lin <jerry@museder.com> 2025年12月6日 凌晨2:05
收件者: "WordPress.org Plugin Directory" <plugins@wordpress.org>

Hi Plugin Review Team,

Thank you for your previous feedback on **Museder RestoreOne**.

I have gone through your comments and the Plugin Check report and pushed a **new version 2.7.40** of the plugin (uploaded on the “Add Your Plugin” page to replace the original submission).

In this version I have:

1. **File handling / AlternativeFunctions**
   - Wrapped all required low-level file functions (fopen, fread, fwrite, fclose, readfile, unlink, chmod, etc.) in clearly documented phpcs:disable/enable blocks.
   - Added explanations that these calls are only used for streaming large backup archives where WP_Filesystem is not reliable or efficient.
   - Where possible, wp_delete_file() is used instead of unlink().
2. **Security: nonce verification and sanitization**
   - Added or centralized check_ajax_referer() / check_admin_referer() for all AJAX and form handlers.
   - Sanitized all $_POST, $_GET and $_FILES inputs using sanitize_text_field(), absint(), sanitize_file_name(), esc_url_raw(), is_uploaded_file(), and array_map() where appropriate.
   - Clarified in comments where the plugin uses HMAC-based tokens instead of nonces (for download links), and that those parameters are validated and do not accept arbitrary user input.
3. **Direct database queries**
   - Reviewed all direct SQL queries and added phpcs:disable/enable annotations with explanations.
   - All table names are taken from $wpdb properties or internal whitelists; no user input is interpolated into SQL.
4. **REST / AJAX permissions and development helpers**
   - Ensured that all REST and AJAX callbacks have proper capability checks or permission callbacks.
   - Guarded calls to set_time_limit() and ini_set() with function_exists() checks and documented them as needed for long-running backup/restore jobs.
5. **Templates and globals**
   - Added PHPCS annotations to template files explaining that the variables are local view context passed from the plugin’s controller and are not exposed as reusable global APIs.
   - All plugin globals and hooks are prefixed with museder_restoreone_.

After these changes I re-ran **WordPress Plugin Check**.
All previously reported *critical* issues (unsafe file I/O, missing nonce checks, unsanitized input, and undocumented SQL) have been addressed.

The remaining findings are:

- A few WordPress.NamingConventions.PrefixAllGlobals.NonPrefixedVariableFound notices where variables are already prefixed with museder_restoreone_ and only used inside the plugin's own templates or handlers.
- Some *warning-level* items for set_time_limit() / ini_set() and direct database queries, which are now documented and guarded as described above.

If you would like me to adjust anything further (for example, refactoring away the remaining globals), I will be happy to make additional changes.

Thank you again for your time and for reviewing Museder RestoreOne.

Best regards,
Jerry Lin

WordPress.org Plugin Directory <plugins@wordpress.org> 於 2025年12月2日週二 上午2:34寫道：

👋 artherslin – let's improve your plugin!

Thank you for submitting your plugin, "Museder RestoreOne".

Before your plugin reaches a human reviewer, our automated tools — which help the team handle around **1,000 plugin reviews per week** — have flagged a few potential issues that you may need to address. To avoid delays and help streamline the manual review process, we've **pended** your submission to give you a chance to review and fix these common issues.

🤖 *This is an automated message generated using a combination of algorithms and AI. It hasn't necessarily been reviewed by a human. Its purpose is to help you resolve potential common issues early, before manual review begins. All AI outputs are marked with the emoji ✨ and it's quite accurate.*

Who are we?

We are a group of volunteers who **help you identify common issues** so that **you can make your plugin more secure, compatible, reliable and compliant** with the guidelines.

For consistency and better communication, your plugin review will be assigned to a single volunteer who will assist you throughout the entire review. However, response times may vary depending on how much time the volunteer is able to contribute to the team and if whether they need to consult something with the rest of the team.

The review process

Please **read this email in full and check each issue**, as well as the links to the documentation and the provided examples. Also, search for any other similar occurrences of the same issue that are not explicitly mentioned in the email.
Make sure you understand the issues so that you can incorporate them into your existing skillset.

	If you decide to continue with the review process, **you must fix any issues, test your plugin, upload a corrected version** and then reply to this email. In case of any doubt, please fix everything else and ask your questions alongside the update.
	Your plugin is **manually checked by a volunteer** who sends you the remaining identified issues in the plugin. **We will be devoting our time to reviewing your plugin**, we ask that you honor this by following the instructions. *Note: Volunteers are not your QA team. They are here to help you identify and understand issues so that you can improve and maintain your plugin in the future. Fixing the issues is your responsibility.*
	If there are no further issues, the plugin will be approved 🎉
	**Be brief and direct in your reply** (please, avoid copy-pasting bloated AI responses, our AI is quite brief), be patient and make sure you have **addressed all the issues and tested your plugin before responding**. *It is disheartening to receive an updated plugin only to find that only a few issues have been resolved, or that it causes a fatal error upon activation.* When not making adequate progress in your review, **it will be delayed and eventually rejected**, for the sake of volunteers devoting their time and other plugin authors who correctly follow the review process. Each volunteer can review up to 300 plugins per week, and they love to have life besides reviewing plugins, so please make things easier for us.

Understanding the Review Queue

When you reply, your plugin enters the review queue again. Fewer review cycles mean quicker approval, while multiple reviews can extend the process to weeks or months.
Tip: Carefully fix all issues and test your plugin before resubmitting to speed up approval.

This process is designed to help you improve your plugin while making the review experience faster and more efficient for everyone.

Have you read the guidelines and this plugin complies with them?

Upon submitting your plugin, you agreed and confirmed that it complies with the WordPress.org Plugin Directory Guidelines, which apply to all plugins in the directory.

Our automated tools have detected patterns that may require a closer look regarding compliance with certain

guidelines. We will verify this during our manual review, but it's best to address any potential issues beforehand. In particular, please pay attention to the following:

- Any included code **must be GPL-compatible**. This means, among others, that the users have to receive the four essential freedoms: (0) to run the program, (1) to study and change the program in source code form, (2) to redistribute exact copies, and (3) to distribute modified versions. *(Guidelines 1 & 4)*
- Plugins **must not restrict or lock functionality**. *(Guidelines 1, 5, & 9 — see clarification on SaaS in Guideline 6)*

Please check it, and if you think everything is fine, do not worry. Our tools are very thorough and may highlight different things as potential issues.

Have you checked for common technical issues?

Please ensure that your plugin adheres to best practices, including the following:

🔴 Proper sanitization of inputs

ℹ️ **Why it matters**: Sanitizing inputs is crucial for protecting against security vulnerabilities like SQL injection, handling invalid data, and ensuring that only safe, expected data is processed.
Please check the official WordPress docs on sanitizing for details — this is a quick summary.

🔍 **Identify unsanitized inputs**: Check any input coming from sources such as: $_GET , $_POST , $_REQUEST , $_COOKIE , $_SERVER , $_SESSION
If these inputs aren't sanitized first, that's a potential risk! 🦹

🛠️ **Fix it**: Always wrap any input with the right sanitization function, like:

Content	Function
Plain text	sanitize_text_field()
Email	sanitize_email()
URL	esc_url_raw()
Key	sanitize_key()
Integer	absint()

Refer to the official WordPress documentation for a complete list of sanitization functions.
👉 Use the most restrictive function that fits the expected content.
👉 Sanitize as early as possible — ideally as soon as the data is received.
👉 Only trust what you've cleaned yourself.

Example:

```
$post_id = (int) $_GET['post_id']; // Sanitized as an integer
$email = sanitize_email( $_GET['email'] ); // Sanitized as an email
```

Your data is now **sanitized**! 🧼🍏

Note: While the json_decode() function in PHP is useful for decoding JSON strings, it does not sanitize the input. Sanitization refers to the process of cleaning or filtering the input data to ensure that it is safe and secure to use.

The json_decode() function simply transforms a JSON string into a PHP array or object. Any potentially malicious

data or scripts may persist after json_decode().
Example(s) from your plugin:

includes/class-restore-handler.php:707 $decoded = json_decode($search_replace_raw, true);
includes/class-ui.php:508 $decoded = json_decode($search_replace_raw, true);
includes/class-settings.php:189 $decoded = json_decode($data, true);
includes/class-ui.php:770 $decoded = json_decode($cloud, true);

... out of a total of 7 incidences.

✔ You can check this using Plugin Check.

🔴 Nonces and User Permissions Before Processing Requests

ℹ️ **Why it matters**: Nonces and permissions checks ensure that the request comes from a trusted source, protecting against security threats like CSRF (Cross-Site Request Forgery) attacks.
Please check the official WordPress docs on nonces and this article for details — this is a quick summary.

🔍 **Spot it**: Look for functions that interact with $_GET , $_POST , $_REQUEST and perform actions triggered by the user/browser that modify data or perform sensitive actions (e.g., privileged actions or data manipulation). For such actions, you should always check for a nonce.
Additionally, verify user permissions if the action is restricted to certain roles (e.g., admin, editor).
When in doubt, play it safe: always check.

🛠️ **Fix it**

1. Create a nonce (wp_nonce_field() , wp_nonce_url() , wp_create_nonce()). If you need to pass the nonce to JavaScript you can make use of wp_localize_script()
2. Pass it with the request.
3. Check the nonce (check_admin_referer() , check_ajax_referer() , wp_verify_nonce()) , and permissions if applicable (current_user_can()) .

Example (nonce check):

```
function musere_save_email(){
    if ( !isset( $_POST['musere_nonce'] ) || !wp_verify_nonce( sanitize_text_field( wp_unslash(
$_POST['musere_nonce'] ) ), 'musere_save_email_action' ) ) {
        wp_send_json_error( 'Invalid nonce.' );
    }

    if ( !current_user_can( 'edit_posts' ) ) {
        wp_send_json_error( 'Insufficient permissions.' );
    }

    if ( isset( $_POST['post_id'] ) && isset( $_POST['price'] ) ) {
        update_post_meta( absint( $_POST['post_id'] ), 'price', floatval( $_POST['price'] ) );
    }
}
add_action( 'wp_ajax_musere_save_email', 'musere_save_email' );
```

⚠️ Without nonce and permissions checks, anyone could call this AJAX endpoint and change the price data for any post - risky!

🔴 Use Prefixes for declarations, globals and stored data

ℹ️ **Why it matters**: Prefixing avoid naming collisions with other themes, plugins, or WordPress core functions.

A prefix is a string placed in front of a name to avoid collisions. **It must be at least 4 characters long, feel distinct and unique to the plugin (do not use common words)**, and be separated by an underscore or dash. *Please check the official WordPress docs on avoiding name collisions.*

🔍 **Identify not prefixed names**: Look for any name that is used in a place where it can create a collision.

Type of element	Affected elements
Declarations	Functions, classes, etc (if not under a namespace)
Globals	Global variables, namespaces, define() .
Data storage	update_option() , set_transient() , update_post_meta() , etc.
WordPress declarations	add_shortcode() , register_post_type() , add_menu_page() , wp_register_script() , wp_localize_script() , add_action('wp_ajax_...') , etc.

If the defined name for that is not prefixed, that's a potential issue! 👮

🛠️ **Fix it**: Always prefix those names, for example if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', $options);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global $musere_options;
- add_action('wp_ajax_musere_save_data', ...);
- namespace artherslin\musederrestoreone;

🔴 Use wp_enqueue commands

ℹ️ **Why it matters**: Because of performance and compatibility, please make use of the built in functions for including static and dynamic JS and/or CSS.

🔍 **Identify JS and CSS outputs**: Look for any <script> or <style> HTML tags in your plugin. In the majority of cases you could enqueue them.

🛠️ **Fix it**: Make use of the specific function for enqueue them:

Type of code	Functions
Static JS	wp_register_script() , wp_enqueue_script() , admin_enqueue_scripts()
Inline JS	wp_add_inline_script()
Static CSS	wp_register_style() , wp_enqueue_style()
Inline CSS	wp_add_inline_style()

👉 In the public pages you can enqueue them using the hook wp_enqueue_scripts() .
👉 In the admin pages you can enqueue them using the hook admin_enqueue_scripts() . You can also use admin_print_scripts() and admin_print_styles() .
👉 As of WordPress 6.3, you can easily pass attributes like defer or async, as of WordPress 5.7, you can pass other attributes by using functions and filters.

Example:

```
function musere_enqueue_script() {
    wp_enqueue_script( 'musere_js', plugins_url( 'inc/main.js', __FILE__ ), array(), MUSERE_VERSION, true );
}
add_action( 'wp_enqueue_scripts', 'musere_enqueue_script' );
```

Your JS/CSS is now **enqueued!**

Possible cases from your plugin include:

templates/page-reports.php:184 <style>
templates/page-pro-retention.php:66 <style>
templates/page-pro-reports.php:101 <script>
templates/page-pro-filters.php:66 <style>
templates/page-pro-filters.php:101 <script>
templates/page-pro-features.php:174 <style>
templates/page-pro-ai.php:112 <script>
templates/page-schedules.php:101 <script>

... out of a total of 15 incidences.

🔴 Trialware and Locked Features

Please review your plugin to ensure that it **does not include any locked or restricted built-in functionality**. This is not permitted under the WordPress.org Plugin Directory Guidelines you agreed to when submitting the plugin.

❌ Guideline 5 — Trialware

Plugins must be fully functional. You may not:

- Lock, disable or limit built-in features behind a license key, trial period, usage limit, time, quota or any other kind of intended restriction.

Even if the locked feature is present in the code "just in case the user upgrades," it's still **not allowed**. Your plugin may point out which features are available through a separated plugin, but that's it. All plugin code hosted on WordPress.org must be **free and fully functional**.

🌐 Guideline 6 — Serviceware

Plugins may connect to a **legitimate external service to perform certain functionality**, provided:

- The service performs actual processing on external servers.
- The functionality provided cannot be done locally by the plugin.
- The service is clearly documented in your readme, including **Terms of Use** and **Privacy Policy** links.

For example: a "Spam checker" plugin that connects to a external service to check for spam (and thus uses it to provide that functionality) is generally acceptable. A plugin that simply checks a license key to unlock local features is not.

✅ Ask yourself:

- Does any function **only work after a license check or payment**?
- Is any functionality in the plugin code **disabled or limited** until it's unlocked?
- Are there any limitations on the plugin **after a certain amount of time or usage**?

After excluding functionalities provided by legitimate external services, if the answer is ***yes*** to any of the above, the

plugin **does not comply**.

🔧 **How to fix it:**

- **Remove all license checks** or other mechanisms that control access to features built in in the plugin code.
- **Remove or fully enable** any built in features that are currently locked or limited.
- Make sure external services are compliant and clearly documented.

ℹ️ **Important clarification:**

WordPress.org is **not a marketplace**. It's a repository for **free, fully functional, GPL-compliant plugins**.

If you are not offering a service and want to offer additional features through a paid version, that code must be:

- **Hosted elsewhere** (e.g., your own website).
- **Not included** in the plugin hosted on WordPress.org.
- **GPL compliant**: Do **not** include any mechanisms that would prevent a plug-in from being used after a license has been checked.

Other details

We've detected some other details that you may want to check.

Review: Missing permission_callback in REST API Route

When using register_rest_route() or wp_register_ability() to define custom REST API endpoints, it is crucial to include a proper permission_callback .

🔒 This callback function ensures that only authorized users can access or modify data through your endpoint.

Code example, checking that the user can change options:

```
register_rest_route( 'museder-restoreone/v1', '/my-endpoint', array(
    'methods' => 'GET',
    'callback' => 'museder-restoreone_callback_function',
    'permission_callback' => function() {
        return current_user_can( 'manage_options' );
    }
) );
```

Please check the register_rest_route() documentation and the current_user_can() documentation.

✅ **When a permission_callback is NOT Required**:

There are valid use cases for public endpoints, such as publicly available data (e.g., posts, public metadata) or endpoints designed for unauthenticated access (e.g., fetching public stats or information).

In these cases, you should use __return_true as the permission_callback to indicate that the endpoint is intentionally public.

🔒 **When a permission_callback IS Required**:

For endpoints that involve sensitive data or actions (e.g., getting not public data, creating, updating, or deleting content).

In these cases, you should always implement proper permission checks.

Possible cases found on this plugin's code:

includes/class-chunk-handler-v2.php:776 register_rest_route('backup-lite/v2', '/prepare', ['methods' => WP_REST_Server::CREATABLE, 'callback' => ['Backup_Lite_Chunk_V2', 'prepare'], 'permission_callback' => '__return_true']);
includes/class-chunk-handler-v2.php:782 register_rest_route('backup-lite/v2', '/chunk', ['methods' => WP_REST_Server::CREATABLE, 'callback' => ['Backup_Lite_Chunk_V2', 'upload_chunk'], 'permission_callback' => '__return_true']);
includes/class-chunk-handler-v2.php:788 register_rest_route('backup-lite/v2', '/finalize', ['methods' => WP_REST_Server::CREATABLE, 'callback' => ['Backup_Lite_Chunk_V2', 'route_finalize'], 'permission_callback' => '__return_true']);

Out of Date Libraries

At least one of the 3rd party libraries you're using is out of date. Please upgrade to the latest stable version for better support and security. We do not recommend you use beta releases.

From your plugin:

assets/vendor/chart.4.4.4.min.js:1 🔴 Chart.js v4.4.4
↳ Possible URL: https://github.com/chartjs/Chart.js

Allowing direct file access to plugin files

Direct file access occurs when someone directly queries a PHP file. This can be done by entering the complete path to the file in the browser's URL bar or by sending a POST request directly to the file.

For files that only contain class or function definitions, the risk of something funky happening when accessed directly is minimal. However, for files that contain executable code (e.g., function calls, class instance creation, class method calls, or inclusion of other PHP files), the risk of security issues is hard to predict because it depends on the specific case, but it can exist and it can be high.

You can easily prevent this by adding the following code at the beginning of all PHP files that could potentially execute code if accessed directly:

```
if ( ! defined( 'ABSPATH' ) ) exit; // Exit if accessed directly
```

Add it after the <?php opening tag and after the namespace declaration, if any, but before any other code.

Example(s) from your plugin:

upload-handler.php:10
download-handler.php:3

👉 Your next steps

This is your checklist:

1. Have you read the guidelines and this plugin complies with them?

2. Have you checked for common technical issues?
3. Other details

If there is something that needs to be fixed, please take your time, fix it and update your plugin files at the "Add your plugin" page, while being logged in with your account "artherslin".
Please be concise and do not list the changes done — we will review the entire plugin again — but do share any clarifications or important context you want us to know.

If after checking the list and do the changes you feel that everything is right or need further clarification, please reply to this email and a volunteer will assist you.

If you believe there is a requirement you cannot accomplish and choose not to make changes, your plugin submission will be rejected after three months.

Thanks!

By taking these steps, you're helping the Plugin Review Team work more efficiently — meaning your plugin (along with the thousands of others in the queue) can be reviewed faster. 🚀 We really appreciate your contribution!

Disclaimers

If, at any time during the review process, you wish to change your permalink (aka the plugin slug) "museder-restoreone", you **must explicitly and clearly tell us what you would like it to be**. Just changing it in your code and in the display name is not sufficient. Remember, permalinks cannot be altered after approval.
This email was partially auto-generated, so please be aware that some information might not be entirely accurate. No personal data was shared with the AI during this process. If you notice any obvious errors or something seems off, feel free to reply — we'll be happy to take a closer look and readjust this automation.

Review ID: AUTOPREREVIEW ❗ LIC museder-restoreone/artherslin/1Dec25/T1 1Dec25/3.7

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/

--
Best Regard

業務專員
葉芸榛 GRACE
M: 0961-353-336

耘業乙創有限公司
YUNYE YICHUAG LTD.
235 新北市中和區安樂路81號2樓
2 F., No. 81, Anle Rd., Zhonghe Dist., New Taipei City 235065, Taiwan
T: 02-2290 0223 FAX:02-2290 0230